МОСКОВСКИЙ ФИЗИКО-ТЕХНИЧЕСКИЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Физтех-школа электроники, фотоники и молекулярной физики

Лабораторная работа
Строки

Кривопалова Полина Б04-501

Долгопрудный, ноябрь 2026

1. Исследование времени работы

1.1. Опыт A

Предположение

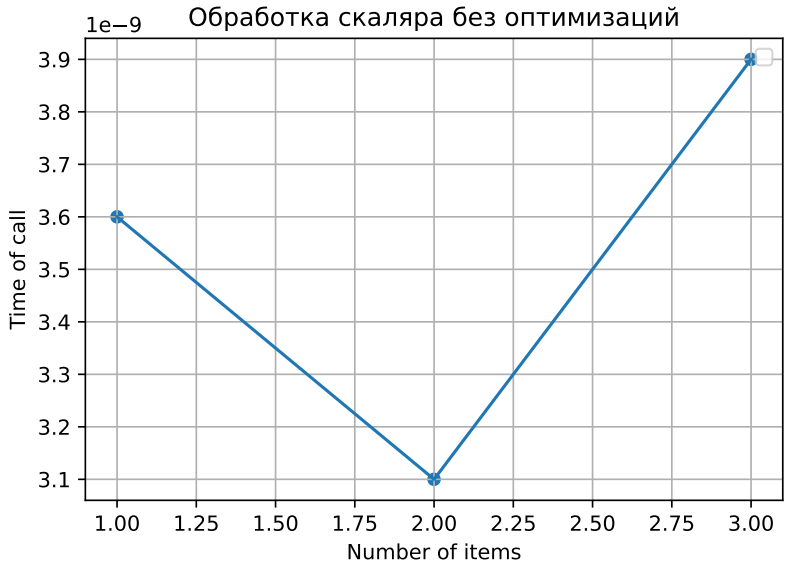
При работе с скалярной величиной время обращения будет минимальным и не зависит от типа обращения.
Обращение к Text будет быстрее, чем к Block и в обоих случаях время обращения по значению будет наибольшим, по ссылке и указателю одинаковым. От размера структур не зависит.
При включении оптимизаций время уменьшится на пару порядков

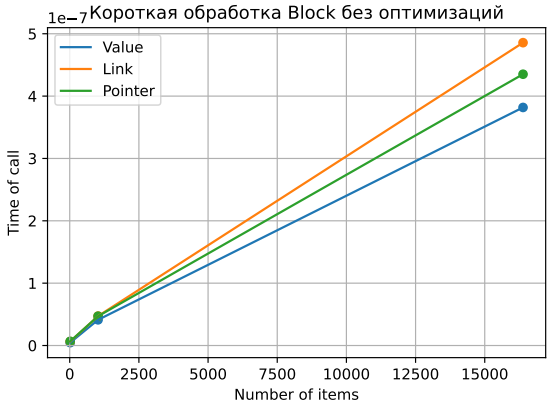
Методика

Измерения проводились при помощи функций аналогичных представленной

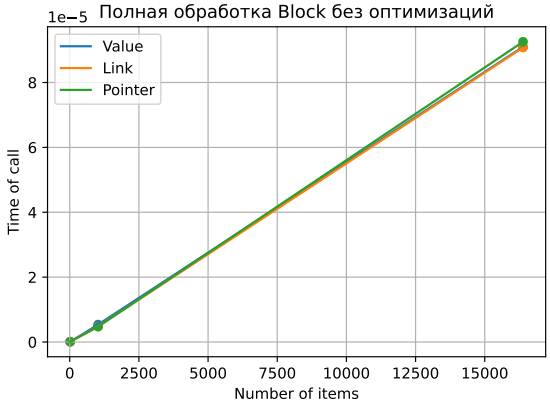
```cpp
double pass_block_16_short(const Block_16 text, int repeats){
    unsigned long long guard = 0;
    auto start = std::chrono::steady_clock::now();
    for (std::size_t r = 0; r < repeats; ++r) {
        block_16_short(text);
    }
    auto finish = std::chrono::steady_clock::now();
    double seconds = std::chrono::duration<double>(finish - start).count();
    return seconds/repeats;
}
```

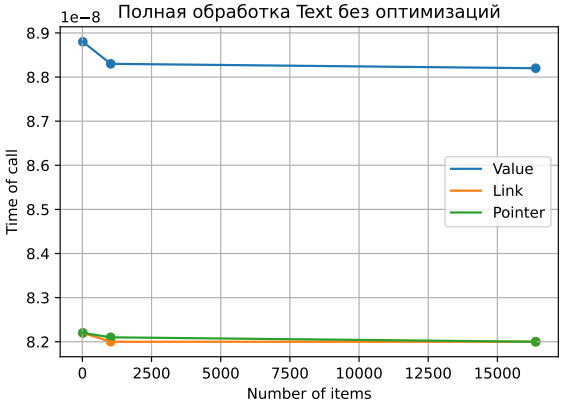
Результаты

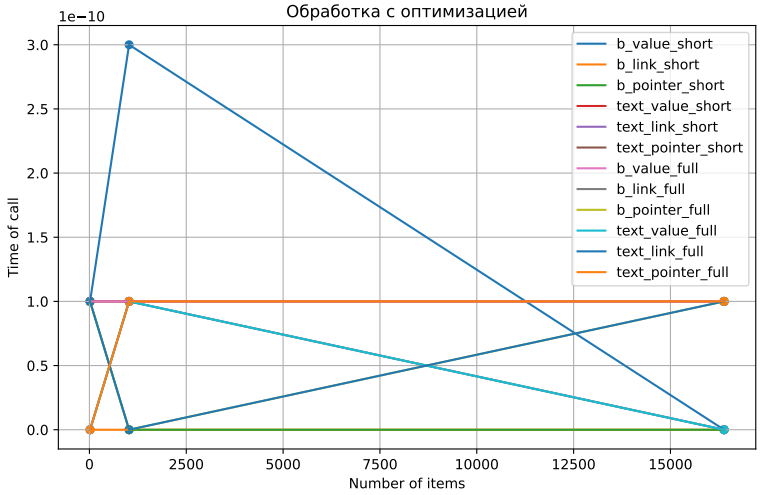












Объяснение результатов

Для скаляра данные похожи больше на случайные выбросы.
При коротком варианте обращение по значению наиболее бстрое для обоих структур. Это может быть связано с тем, что когда передаётся ссылка и указатель происходит дополнительный шаг расшифровки, что замедляет работу.
При полной обработке для Block разницы нет. А для Text наоборот: при обрашении по значению время наибольшее. Причиной может быть то, что
Разброс значений при оптимизации скорее случайный, чем показательный

1.2. Опыт B

Предположение

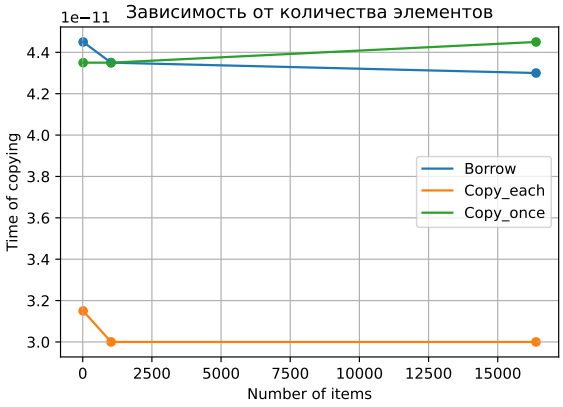
При анализировании исходного текста время будет минимальным. Многократное копирование будет самым долгим. Эта тенденция будет наблюдаться при обоих параметрах сравнения.

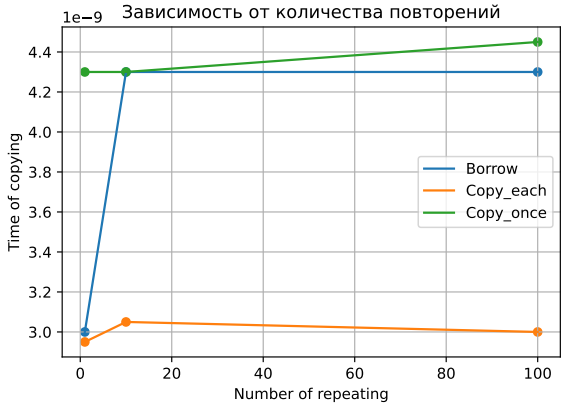
Методика

Измерения проводились при помощи функций аналогичных представленной

```cpp
double repeat (const Text text, int k, void (*func)(const Text, int), int G, int repeats){
    double seconds;
    for (int j = 0; j < repeats; ++j){
        auto start = std::chrono::steady_clock::now();
        for (int i = 0; i < G; ++i) {
            func(text, k);
        }
        auto finish = std::chrono::steady_clock::now();
        seconds += std::chrono::duration<double>(finish - start).count();
    }
    return seconds/(repeats * G * k);
}
```

Результаты





Объяснение результатов

С использованием оптимизаций разница в результатах больше похожа на случайный разброс

2. Исследование работы с памятью

Предположение

Методика

Исследование проводилось с помощью вспомогательной структуры *mem* и функции

mode	allocations	copied_bytes	extra_bytes
borrow	0	0	0
copy_each	$k \cdot G$	$(L-1) \cdot G \cdot k$	L
copy_once	G	$(L-1) \cdot G$	L

Таблица 3: Предсказание результатов

```
mem repeat_mem (const Text text, int k, mem (*func)(const Text, int), int G){
    int counter;
    int copys;
    int elem;
    for (int i = 0; i < G; ++i) {
        mem a = func(text, k);
        counter += a.counter;
        copys += a.copys;
        elem = a.elem;
    }
    return {counter, copys, elem};
}
```

Результаты

Объяснение результатов

Все предположения оказались верными

length	mode	repeats	uses	allocations	copied_bytes	extra_bytes
15	borrow	100	1	0	0	0
1023	borrow	100	1	0	0	0
16383	borrow	100	1	0	0	0
15	copy_each	100	1	100	1500	16
1023	copy_each	100	1	100	102300	1024
16383	copy_each	100	1	100	1638300	16384
15	copy_once	100	1	100	1500	16
1023	copy_once	100	1	100	102300	1024
16383	copy_once	100	1	100	1638300	16384
15	borrow	100	10	0	0	0
1023	borrow	100	10	0	0	0
16383	borrow	100	10	0	0	0
15	copy_each	100	10	1000	15000	16
1023	copy_each	100	10	1000	1023000	1024
16383	copy_each	100	10	1000	16383000	16384
15	copy_once	100	10	100	1500	16
1023	copy_once	100	10	100	102300	1024
16383	copy_once	100	10	100	1638300	16384
15	borrow	100	100	0	0	0
1023	borrow	100	100	0	0	0
16383	borrow	100	100	0	0	0
15	copy_each	100	100	10000	150000	16
1023	copy_each	100	100	10000	10230000	1024
16383	copy_each	100	100	10000	163830000	16384
15	copy_once	100	100	100	1500	16
1023	copy_once	100	100	100	102300	1024
16383	copy_once	100	100	100	1638300	16384